

[TÍTULO POR DEFINIR]

Arturo Cantú Olivares¹, Diego Sebastian Cruz Cervantes¹,
Luis Fernando Maldonado¹, Jesús Gabriel Gudiño Lara¹, Daniel Cantón
Enriquez¹, and [Colaboradores Terraquea]²

¹ Universidad Autónoma de Querétaro (UAQ), Querétaro, México
dcruz54@alumnos.uaq.mx

² Terraquea, México
[correo@terraquea.com]

Abstract. [PENDIENTE — redactar al final cuando los resultados finales estén completos.]

Estructura sugerida (4–6 oraciones):

1. Contexto: la detección de árboles individuales (ITD) en nubes de puntos LiDAR UAV de alta densidad (ULS) es fundamental para inventarios forestales automatizados a escala.
2. Problema: no está claro si el preprocesado semántico de la nube de puntos es el factor determinante en la calidad de la detección de instancias individuales.
3. Método: comparamos tres estrategias de preprocesado — sin preprocesado, Random Forest sobre features geométricas y PointNet++ MSG — seguidas de un segmentador volumétrico Watershed 3D idéntico calibrado por flujo, evaluadas sobre FOR-instance V1 en cinco tipos de bosque.
4. Resultado clave: [PENDIENTE].
5. Conclusión: [PENDIENTE].

Keywords: Individual Tree Segmentation · UAV laser scanning (ULS) · Segmentación semántica · Watershed 3D · Random Forest · PointNet++ · FOR-instance

1 Introducción

1.1 Motivación

Los inventarios forestales tradicionales dependen de mediciones de campo costosas, de baja cobertura espacial y de actualización infrecuente. El UAV laser scanning (ULS) combina la perspectiva aérea del LiDAR aerotransportado con densidades de puntos uno o dos órdenes de magnitud superiores al ALS convencional ($\sim 7,000$ pts/m² frente a ~ 5 pts/m²). Esta ultra-alta densidad permite caracterizar el bosque a nivel de árbol individual con gran detalle, incluyendo no solo copas superiores, sino también vegetación baja, troncos, ramas y suelo. Como consecuencia, la segmentación automática de árboles individuales en datos

ULS sigue siendo un problema abierto por el ruido potencial en modelos de altura, la ambigüedad entre puntos de copa y vegetación subyacente, y la variabilidad estructural entre tipos de bosque.

[PENDIENTE EXPANDIR — 2–3 párrafos adicionales sobre la importancia del problema y el estado general del campo]

1.2 Problema específico

Un pipeline típico de Individual Tree Segmentation (ITS) sobre datos ULS de alta densidad comprende dos etapas en cascada: (1) preprocesado semántico, que separa los puntos de árbol de los de suelo y vegetación no arbórea, y (2) segmentación de instancias individuales, que asigna cada punto de árbol a un identificador de árbol único. En trabajos previos, la calidad del preprocesado semántico suele reportarse como un factor habilitador del rendimiento de instancia, pero rara vez se aísla experimentalmente su contribución frente al segmentador [2, 5, 1]. Esta suposición no ha sido evaluada sistemáticamente en datos ULS de alta densidad sobre múltiples tipos de bosque mediante un experimento controlado donde el segmentador de instancias sea idéntico y solo varíe el preprocesado.

1.3 Contribuciones

Este trabajo presenta las siguientes contribuciones:

1. Una comparación sistemática de tres estrategias de preprocesado semántico — nube cruda sin preprocesado, Random Forest sobre features geométricas y PointNet++ MSG — sobre el benchmark FOR-instance V1 [1], que cubre cinco tipos de bosque en cuatro continentes, con un segmentador de instancias Watershed 3D idéntico y calibrado de forma independiente para cada flujo.
2. Un análisis de la relación entre la calidad del preprocesado semántico (tree IoU punto a punto) y la calidad de la segmentación de árboles individuales (F1 con IoU 3D ≥ 0.5), que permite identificar en cuál de las dos etapas se concentra el error del pipeline.
3. Evidencia sobre la viabilidad de Random Forest con features geométricas como alternativa al aprendizaje profundo para preprocesado semántico en pipelines ITS sobre datos ULS de alta densidad, sin requerir GPU ni arquitectura profunda.

1.4 Estructura del paper

El resto del paper se organiza de la siguiente manera. La Sección 2 revisa el trabajo relacionado. La Sección 3 describe el dataset utilizado. La Sección 4 detalla los tres pipelines propuestos y el detector compartido. La Sección 5 presenta el protocolo de evaluación. La Sección 6 reporta y analiza los resultados. La Sección 7 discute las implicaciones y limitaciones. La Sección 8 concluye.

2 Trabajo Relacionado

- 2.1 Detección de árboles individuales en LiDAR aerotransportado
- 2.2 Detección de árboles individuales en LiDAR denso (ULS / HD-ALS)
- 2.3 Clasificación semántica de nubes de puntos forestales
- 2.4 Benchmarks de segmentación forestal

3 Dataset

3.1 FOR-instance

El dataset FOR-instance [1] es un benchmark de acceso abierto diseñado para la evaluación de métodos de segmentación semántica e de instancias de árboles individuales en nubes de puntos LiDAR de alta densidad. La versión V1 comprende seis colecciones (NIBIO, NIBIO2, CULS, TUWIEN, RMIT, SCION) de datos adquiridos con escáneres láser Riegl (VUX-1 UAV y Mini-VUX) montados en vehículos aéreos no tripulados (UAV laser scanning, ULS) sobre sitios geográficamente diversos que cubren tipos de bosque representativos a nivel global.

En este trabajo se utilizan las cinco colecciones que cubren tipos de bosque distintos (NIBIO, CULS, TUWIEN, RMIT, SCION). Se excluye explícitamente NIBIO2 por una razón metodológica: NIBIO2 corresponde a parcelas adicionales del mismo tipo de bosque boreal conífero noruego (*Picea abies*) muestreado por NIBIO. Incluir ambas colecciones sobre-representaría el bosque boreal en el conjunto de evaluación, sesgando las métricas agregadas hacia un único tipo estructural y diluyendo la señal experimental sobre la generalización entre tipos de bosque, que es objetivo central del estudio.

Las nubes de puntos presentan densidades que varían fuertemente entre colecciones: medidas sobre los 32 plots empleados en este trabajo, el rango global es 454–10 156 pts/m² (mediana por colección: RMIT 473, TUWIEN 1 406, CULS 2 510, SCION 3 064, NIBIO 7 246), todas ellas considerablemente más altas que el LiDAR aerotransportado convencional (~ 5 pts/m²) y consistentes con los rangos reportados por Puliti et al. [1]. Esta alta densidad permite la anotación manual precisa de instancias individuales. Cada punto está anotado con una etiqueta semántica de entre seis clases — suelo, vegetación baja, tronco, ramas vivas, ramas muertas y puntos de borde (outpoints) — así como con un identificador de árbol individual (treeID) para todos los árboles del dosel anotados.

3.2 Colecciones y tipos de bosque

El dataset agrupa los plots en cinco colecciones geográficas, cada una representando un tipo de bosque distinto (Tabla 1).

La diversidad geográfica y estructural de las colecciones es relevante para evaluar la generalización de los métodos: NIBIO representa el 50% del conjunto de test y corresponde a bosque boreal de alta densidad, mientras que RMIT y TUWIEN representan estructuras de copa radicalmente diferentes entre sí y respecto al bosque boreal.

Table 1. Colecciones del dataset FOR-instance V1 utilizadas en este trabajo.

Código	País	Tipo de bosque	Plots test	Árboles GT
NIBIO	Noruega	Boreal conífero (<i>P. abies</i>)	6	161
CULS	Rep. Checa	Templado conífero (<i>P. sylvestris</i>)	1	20
TUWIEN	Austria	Mixto deciduo aluvial	1	35
RMIT	Australia	Esclerófilo seco (eucaliptos)	1	64
SCION	Nueva Zelanda	Plantación conífera (<i>P. radiata</i>)	2	43
Total			11	323

3.3 Splits y desdoblamiento interno

El dataset prescribe un split oficial: 70% de los plots para desarrollo (dev) y 30% para evaluación (test), con todos los sitios geográficos representados en ambas particiones. En este trabajo se utiliza el split oficial sin modificaciones. El conjunto dev se emplea para entrenamiento de modelos (Flujos B y C) y calibración de hiperparámetros del detector (los tres flujos). El conjunto test se utiliza exclusivamente para reportar las métricas finales y no interviene en ninguna decisión de diseño o calibración.

Desdoblamiento interno dev $\rightarrow$ train+val. Dado que FOR-instance V1 define exclusivamente las particiones dev y test en su metadata oficial, el conjunto dev se subdivide internamente en un conjunto de entrenamiento (80% de los plots) y un conjunto de validación interno (20% restante) mediante una partición aleatoria a nivel de plot con semilla fija (`random_state = 42`) para reproducibilidad. La partición se realiza a nivel de archivo `.las` y no a nivel de punto para evitar *data leakage* espacial entre train y val, siguiendo el criterio adoptado por ForAINet [15] y SegmentAnyTree [16]. El conjunto val interno se utiliza para: (i) monitorear la pérdida durante el entrenamiento de PointNet++ y guiar el scheduler, (ii) ejecutar el grid search de los hiperparámetros del Watershed 3D de forma independiente por flujo, y (iii) reportar las métricas val que se presentan en la Sección 6. El conjunto test oficial de FOR-instance permanece intacto y se reserva exclusivamente para el reporte de métricas finales, sin intervenir en ninguna decisión de diseño o calibración.

Reproducibilidad. Para garantizar reproducibilidad, todas las fuentes de aleatoriedad del pipeline — partición train/val interna, submuestreo por plot del Random Forest, inicialización y barajado del entrenamiento de PointNet++, e inferencia multi-pasada de PointNet++ — usan la semilla fija 42 salvo indicación explícita en contrario. En el caso del submuestreo por plot del Random Forest, la semilla efectiva se deriva de forma determinística del nombre del plot vía CRC32, de modo que cada plot reciba siempre el mismo subconjunto de puntos con independencia del orden de iteración.

3.4 Ground truth para segmentación de instancias

Para la tarea de segmentación de árboles individuales el ground truth está definido a nivel de punto: cada punto de la nube tiene asociado un identificador entero `treeID` que indica a qué árbol individual pertenece (o cero para puntos no arbóreos). El protocolo de evaluación compara el conjunto de puntos predicho para cada instancia contra el conjunto de puntos GT del `treeID` mediante IoU 3D punto a punto (Sección 5). Los puntos clasificados como *outpoint* en FOR-instance — correspondientes a árboles parcialmente observados en los bordes de los plots — se excluyen de la evaluación tanto para predicciones como para ground truth.

4 Metodología

4.1 Visión general del pipeline

Los tres flujos propuestos comparten una arquitectura de dos etapas en cascada. La Etapa 1 es el preprocesado semántico, cuyo objetivo es separar los puntos de árbol del resto de la nube (suelo, vegetación baja, ramas caídas). La Etapa 2 es el segmentador de instancias, que opera sobre los puntos clasificados como árbol y asigna a cada uno un identificador de árbol individual. La única variable entre flujos es la Etapa 1. La Etapa 2 es idéntica en arquitectura para los tres flujos — un Watershed 3D volumétrico sobre la nube normalizada por altura sobre el suelo (HAG) — con parámetros calibrados mediante grid search independiente por flujo sobre el split val maximizando el F1 de instancia (umbral IoU 3D ≥ 0.5).

Esta calibración independiente garantiza que cada flujo opere bajo sus condiciones óptimas, de modo que las diferencias en F1 de instancia entre flujos reflejen el efecto del preprocesado semántico y no diferencias en la configuración del segmentador.

Los tres flujos son:

- **Flujo A:** ninguno (nube cruda completa).
- **Flujo B:** Random Forest sobre features geométricas (ML clásico).
- **Flujo C:** PointNet++ MSG (aprendizaje profundo).

Los tres flujos operan sobre la nube completa del plot. Ningún flujo divide la nube en bloques: el clasificador de cada flujo se evalúa punto a punto sobre la nube entera, sin ventanas deslizantes ni reproyección, y el segmentador de instancias opera sobre el subconjunto de puntos resultante (todos los puntos válidos para el Flujo A; los clasificados como árbol para los Flujos B y C).

4.2 Flujo A — Sin preprocesado (baseline)

El Flujo A no aplica ningún clasificador semántico. La nube de puntos completa del plot, conservando todos los puntos con etiqueta de clasificación de FOR-instance en el conjunto $\{1, 2, 4, 5, 6\}$ (suelo y vegetación baja no arbórea, y todas

las variantes de árbol), se pasa directamente al segmentador Watershed 3D de la Etapa 2. Los puntos con clase $\in \{0, 3\}$ (*never-classified* y *outpoint*, estos últimos correspondientes a árboles parcialmente observados en los bordes del plot) se excluyen en todos los flujos por protocolo, tanto en el preprocesado como en la evaluación de instancias, siguiendo la especificación oficial de FOR-instance. Este flujo establece el nivel de rendimiento de instancia alcanzable sin ninguna separación semántica previa y sirve como referencia para cuantificar el valor añadido de los clasificadores de los Flujos B y C.

Preprocesado común de altura sobre el suelo (HAG). Aunque el Flujo A no tiene un clasificador semántico, comparte con los Flujos B y C el mismo preprocesado geométrico de altura sobre el suelo, necesario para que el segmentador Watershed 3D opere en coordenadas invariantes al relieve (Sección 4.5). El DTM utilizado para calcular la HAG se construye por plot tomando la mediana de z por celda sobre los puntos clasificados como suelo (clase 2 de FOR-instance), rasterizado a una resolución horizontal de 0,5 m. Las celdas sin puntos de suelo se rellenan por interpolación bilineal (`scipy.interpolate.griddata`, método `linear`), con *fallback* al vecino más cercano para las celdas de borde que queden fuera del casco convexo de los puntos de suelo. Posteriormente se aplica un suavizado gaussiano cuyo σ equivale a una ventana 5×5 ($\sigma \approx 2,5$ celdas) para reducir artefactos locales. Tras calcular la HAG, el canal se trunca al rango $[0, h_{\max}]$ con $h_{\max} = 50$ m: los valores negativos (ruido del DTM bajo el suelo) se clampean a cero y los valores superiores se saturan a 50 m, un umbral que cubre con holgura las alturas máximas observadas en los cinco sitios de FOR-instance V1. Para el Flujo C (PointNet++) este canal HAG se renormaliza adicionalmente al intervalo $[0, 1]$ dividiendo por $h_{\max}$, de modo que sea comparable con la intensidad láser (también en $[0, 1]$) como feature de entrada a la red.

4.3 Flujo B — Random Forest sobre features geométricas

Features. Cada punto se representa mediante un vector de 27 features geométricas: 13 descriptores dependientes de escala calculados a dos vecindades ($k = 20$ y $k = 50$ vecinos), más una feature adicional de densidad volumétrica compartida entre escalas ($13 \times 2 + 1 = 27$). La selección sigue el catálogo canónico de Weinmann et al. [12], complementado con descriptores verticales basados en la altura sobre el suelo (HAG) que han demostrado ser predictores dominantes de la separación árbol/no-árbol en bosques densos [13, 14].

Por cada vecindad de tamaño k se calculan ocho descriptores eigen-basados a partir de los autovalores normalizados $\lambda_1 \geq \lambda_2 \geq \lambda_3$ de la matriz de covarianza local (estimada con divisor k , no $k - 1$, siguiendo la convención de Weinmann et al. [11]) — linealidad, planaridad, esfericidad, omnivarianza, anisotropía, eigentropía, suma de eigenvalores y cambio de curvatura — más cinco descriptores locales: HAG del punto central, verticalidad (definida como $1 - |\langle \mathbf{n}, [0, 0, 1] \rangle|$ con $\mathbf{n}$ la normal local), rugosidad ($\text{std}(z)$ entre vecinos), rango vertical ($\max(z) - \min(z)$ entre vecinos) e intensidad normalizada del retorno láser. La feature restante — densidad volumétrica local — es independiente de la escala k y se calcula una

Table 2. Catálogo de features geométricas para el clasificador Random Forest (Flujo B). 13 descriptores dependientes de escala calculados a dos vecindades $k = 20$ y $k = 50$, más 1 feature de densidad volumétrica compartida entre escalas, producen un vector total de $13 \times 2 + 1 = 27$ dimensiones por punto. En las filas 1-6 y 8 los λ_i denotan los autovalores normalizados $\lambda_i/(\lambda_1 + \lambda_2 + \lambda_3)$, siguiendo la convención de Weinmann et al. [11]; la fila 7 (suma de eigenvalores) usa los autovalores crudos.

# Feature	Fórmula / definición
<i>Por escala $k \in \{20, 50\}$</i>	
1 Linealidad	$(\lambda_1 - \lambda_2)/\lambda_1$
2 Planaridad	$(\lambda_2 - \lambda_3)/\lambda_1$
3 Esfericidad	λ_3/λ_1
4 Omnivarianza	$(\lambda_1 \lambda_2 \lambda_3)^{1/3}$
5 Anisotropía	$(\lambda_1 - \lambda_3)/\lambda_1$
6 Eigenentropía	$-\sum_i (\lambda_i / \sum_j \lambda_j) \ln(\lambda_i / \sum_j \lambda_j)$
7 Suma de eigenvalores	$\lambda_1 + \lambda_2 + \lambda_3$
8 Cambio de curvatura	$\lambda_3/(\lambda_1 + \lambda_2 + \lambda_3)$
9 HAG	$z - \text{DTM}(x, y)$
10 Verticalidad	$1 - \langle \mathbf{n}, [0, 0, 1] \rangle $
11 Rugosidad	$\text{std}(z_{\text{vecinos}})$
12 Rango vertical	$\max(z) - \min(z)$ entre vecinos
13 Intensidad normalizada	$I/I_{\max}$ por plot
<i>Compartida entre escalas</i>	
14 Densidad volumétrica local	$ \{p_j : \ p_j - p_i\ \leq r\} $ (cardinalidad de la bola de radio r)

sola vez por punto como la cardinalidad de la bola de radio fijo r centrada en el punto. Las dos escalas k proporcionan contexto fino y grueso simultáneamente, siguiendo la estrategia multi-escala de Weinmann et al. [11].

Para evitar que en plots de baja densidad (p. ej. RMIT, mediana ~ 473 pts/m²) los vecinos queden demasiado alejados y contaminen los descriptores locales con contexto irrelevante, se impone un radio máximo de búsqueda de 5,0 m: los vecinos a mayor distancia dentro de los k más próximos se descartan, y los puntos con menos de cuatro vecinos válidos tras el corte reciben *features* en cero para esa escala (valor neutro para el Random Forest). Este régimen afecta a una fracción muy reducida de puntos en los plots considerados y se reporta explícitamente para trazabilidad.

Nota sobre la feature de densidad volumétrica. La feature de densidad volumétrica (fila 14) se computa como la cardinalidad real del conjunto $\{p_j : \|p_j - p_i\| \leq r\}$ con $r = \text{density_radius} = 0,5$ m, idéntico para todos los plots. A diferencia de los descriptores eigen-basados, no depende de la vecindad k : se calcula una sola vez por punto mediante una consulta de bola al `KDTree` global del plot (`ckdTree.query_ball_point`), de modo que su valor no satura por arriba en zonas de alta densidad. Se reporta en forma de conteo entero para coincidir con la convención de *neighborhood cardinality* de Weinmann et al. [11]; al ser una

Table 3. Hiperparámetros del clasificador Random Forest (Flujo B).

Hiperparámetro	Valor	Criterio de selección
n_estimators	200	Estabilización curva OOB
max_depth	None	Default Breiman [5]
max_features	sqrt	Reducción de varianza [5]
class_weight	balanced	Desbalance árbol/no-árbol
max_samples_per_plot	12,000	Submuestreo por plot, equilibrio clases y sitios

transformación lineal de la densidad volumétrica $|\cdot|/V_{\text{esfera}}$, el factor $1/V_{\text{esfera}}$ es absorbido por los umbrales de split del Random Forest sin afectar las decisiones.

Hiperparámetros y entrenamiento. El clasificador es un `RandomForestClassifier` de scikit-learn entrenado a nivel de plot (no por bloques): los 27 features se computan sobre la nube completa de cada plot del split train, los puntos se etiquetan como árbol/no-árbol según el ground truth de FOR-instance, y se aplica un submuestreo balanceado *por plot* que retiene hasta 12 000 puntos por clase en cada plot. Este esquema equipara la contribución de cada sitio al conjunto final de entrenamiento, evitando que los sitios con plots grandes (NIBIO) dominen el modelo. La alternativa de submuestrear globalmente tras concatenar todos los plots introduciría un sesgo proporcional al número de puntos por sitio, contrario al objetivo de evaluar generalización entre tipos de bosque que motiva el presente trabajo. El submuestreo es determinístico: la semilla efectiva de cada plot se deriva de forma determinística del nombre del archivo vía `CRC32` sobre la semilla global `random_state=42`, de forma que cada plot siempre recibe los mismos puntos con independencia del orden de iteración. El modelo se entrena con `n_estimators=200` árboles, sin límite de profundidad (`max_depth=None`) y con `class_weight=balanced` para compensar el desbalance residual.

La elección `max_depth=None` corresponde al ajuste por defecto de Random Forest según Breiman [5]: cada árbol crece hasta la pureza de la hoja, y el control de varianza recae sobre el promediado del bagging y la aleatorización de features (`max_features =  $\sqrt{p}$`). Limitar la profundidad introduciría sesgo sin un beneficio claro de generalización para la dimensionalidad y el tamaño de muestra empleados. La elección `n_estimators=200` se validó empíricamente como punto de estabilización de la curva Out-Of-Bag.

Inferencia. En inferencia el clasificador se aplica directamente a la nube completa del plot: los 27 features se computan punto a punto y la predicción binaria se obtiene aplicando el modelo entrenado, sin agregación por bloques ni promediado de probabilidades.

4.4 Flujo C — PointNet++ MSG

Arquitectura. Se emplea la arquitectura PointNet++ [2] en su variante *Multi-Scale Grouping* (MSG) para segmentación semántica punto a punto, con cuatro

Table 4. Configuración de PointNet++ MSG (Flujo C).

Parámetro	Valor
Variante	MSG (Multi-Scale Grouping)
Módulos Set Abstraction	4
Módulos Feature Prop.	4
Puntos de entrada	8,192 por plot
Dimensión de entrada	5 ($XYZ + \text{HAG_norm} + \text{intensidad}$)
Número de clases	2 (árbol / no-árbol)
Función de pérdida	NLLLoss ponderada por clase
Optimizador	Adam, $\text{lr} = 0.001$, $\text{weight_decay} = 10^{-4}$
Scheduler	StepLR, $\gamma = 0.7$ cada 20 épocas
Gradient clipping	$\ g\ _2 \leq 1,0$ (norma global)
Dropout (cabeza FP)	$p = 0,5$
Batch size	16
Épocas	100
Mixed precision	Sí (<code>torch.amp</code> , <code>bfloat16</code>)
Hardware	Apple M4 Pro (memoria unificada)

módulos de agrupamiento jerárquico (Set Abstraction) seguidos de cuatro módulos de propagación de features (Feature Propagation). La implementación toma como base el repositorio público de Xu Yan [4]. Los radios de búsqueda de las capas *Set Abstraction* operan en el *espacio normalizado* del sample (tras dividir las coordenadas por el radio máximo del bloque de 8,192 puntos), no en metros absolutos; esto hace que la escala efectiva de cada radio dependa del tamaño físico del plot. La red recibe como entrada nubes submuestreadas a 8,192 puntos por plot, donde cada punto se representa por un vector de cinco dimensiones: las tres coordenadas XYZ normalizadas, la altura sobre el suelo HAG normalizada y la intensidad del retorno láser. La cabeza de clasificación punto a punto consiste en una capa lineal $128 \rightarrow 64$ con `BatchNorm`, `ReLU` y *dropout* con $p = 0,5$, seguida de una capa lineal $64 \rightarrow C$ con $C = 2$ clases. La salida son log-probabilidades por punto sobre las dos clases (árbol / no-árbol).

Justificación de la entrada de 5 dimensiones. La elección de $XYZ + \text{HAG} + I$ como entrada se basa en mantener *consistencia con las señales crudas disponibles para los otros flujos*. El Flujo B (RF) consume HAG e intensidad como features de primer orden; usar las mismas señales en el Flujo C garantiza que la comparación entre RF y PointNet++ aísle el efecto del modelo (features manuales vs. aprendidas) y no el efecto de la información disponible en la entrada. Esta es la única forma honesta de comparar dos clasificadores semánticos en condiciones equivalentes: ambos ven los mismos canales crudos. Esta elección es además consistente con Wang et al. [17], que reporta mejoras significativas al incorporar HAG e intensidad en PointNet++ para clasificación forestal.

Entrenamiento. La pérdida es `NLLoss` con pesos $w_c = N/(2N_c + \epsilon)$, donde N es el total de puntos de entrenamiento, N_c el conteo de la clase c y $\epsilon = 10^{-6}$, para compensar el desbalance arbórea/no-arbórea. Se optimiza con Adam (lr inicial 0,001, weight decay 10^{-4}). El scheduler reduce la tasa de aprendizaje un factor $\gamma = 0,7$ cada 20 épocas, hasta 100 épocas en total. Para estabilizar el entrenamiento bajo *mixed precision* se aplica *gradient clipping* global por norma ℓ_2 con umbral 1,0 (`torch.nn.utils.clip_grad_norm_`).

La augmentación incluye tres operaciones aplicadas por muestra tras la normalización per-sample del bloque de 8,192 puntos (centrado en el centroide y división por el radio máximo): (i) *jitter* gaussiano independiente por coordenada con $\sigma = 0,02$ en el *espacio normalizado* — equivalente a una fracción pequeña del radio máximo del sample y no a una distancia fija en metros —, (ii) reflexión aleatoria sobre el eje X con probabilidad 0,5, aprovechando la simetría horizontal aproximada de las copas forestales, y (iii) escalado isotrópico uniforme en $[0,9, 1,1]$. No se aplica rotación en Z porque la orientación del grid del dataset es consistente entre plots. El batch size se fija a 16 aprovechando la memoria unificada del Apple M4 Pro, y se emplea *mixed precision* en `bfloat16` (sin *loss scaling*, ya que `bfloat16` preserva el rango de exponente de `float32`) para acelerar el entrenamiento.

Decisión de entrenamiento único. PointNet++ se entrena con una única configuración de hiperparámetros, sin búsqueda exhaustiva. El objetivo del paper no es encontrar la configuración óptima de PointNet++ sino evaluar el valor del aprendizaje profundo como estrategia de preprocesado en condiciones representativas de uso real. En aplicaciones forestales aplicadas, un usuario que adopta PointNet++ entrena el modelo con una configuración estándar documentada en la literatura, no mediante optimización intensiva. Evaluar PointNet++ bajo esas condiciones es más informativo para la comunidad que reportar su mejor resultado teórico posible.

Inferencia. PointNet++ fue entrenado con submuestreo aleatorio global del plot completo a $N_s = 8\,192$ puntos por muestra y normalización per-sample por centroide y radio máximo. Previo a la normalización per-sample, las coordenadas XY se centran en el centroide del plot para reducir el error numérico de `float32` sobre coordenadas UTM (esta traslación global no altera el resultado de la normalización per-sample posterior, que absorbe cualquier desplazamiento rígido). La inferencia replica exactamente esta estrategia para garantizar consistencia entre train y test: cada plot se procesa mediante n_{passes} submuestreos aleatorios independientes, con

$$n_{\text{passes}} = \min(1000, \max(50, \lfloor 3N/N_s \rfloor)), \quad (1)$$

donde N es el número de puntos válidos del plot. En cada pasada, los N_s puntos seleccionados se pre-centran y normalizan como en entrenamiento, y las log-probabilidades por punto se acumulan. Tras todas las pasadas, cada punto recibe la media de las probabilidades predichas en las pasadas en que fue seleccionado,

y la clase final se obtiene como $\arg \max$. Con este esquema la cobertura efectiva supera el 99% de los puntos del plot en todos los casos observados. Los puntos residuales no cubiertos se asignan por interpolación KNN ponderada por el inverso de la distancia ($w_j \propto 1/d_j$) con $k = 3$ desde los puntos ya cubiertos más cercanos.

La alternativa de una única pasada con extrapolación 1-NN fue descartada por dos razones: (a) introduciría un *mismatch de escala* entre train y test, ya que los radios de las capas *Set Abstraction* están calibrados a la escala del sample normalizado de $N_s = 8192$ puntos tras la división por el radio máximo del bloque, y (b) en plots típicos etiquetaría alrededor del 99% de los puntos por vecindad y no por predicción directa de la red, degradando la calidad semántica sin justificación.

4.5 Segmentador de instancias — Watershed 3D volumétrico

Justificación del método. La densidad ultra-alta de los datos ULS ($\sim 7,000$ pts/m²) permite ejecutar el paso de *fill* del watershed directamente sobre una grilla 3D de densidad, preservando la estructura vertical de la nube. Un watershed 2D tradicional sobre un Canopy Height Model (CHM) asigna cada punto en función de su proyección XY , lo que produce errores sistemáticos cuando puntos de tronco o de rama baja caen bajo la proyección horizontal de una copa vecina. En este trabajo se emplea en cambio un *Watershed 3D volumétrico* en el que las semillas se detectan sobre la *envolvente superior del volumen* — equivalente a un CHM derivado localmente del propio grid 3D de densidad — pero el *fill* del watershed se ejecuta sobre la grilla 3D con una máscara de vóxeles ocupados, de modo que cada punto es asignado al árbol cuyo vóxel tridimensional lo contiene y no según su mera proyección horizontal. Esta distinción es crítica: el verdadero aporte del enfoque 3D no es el seeding (los máximos locales de densidad 3D caerían en los troncos y ramas gruesas, no en los ápices, arruinando la segmentación; por eso la literatura ITS estándar usa envolventes tipo CHM para seeding incluso con fill 3D [9]), sino el fill volumétrico, que es lo que realmente separa troncos adyacentes y puntos de sotobosque.

Uso de HAG como eje vertical. Una característica clave del segmentador es que reemplaza la coordenada Z absoluta por la altura sobre el suelo (Height Above Ground, HAG) antes de voxelizar la nube. La HAG se calcula sustrayendo a cada punto la altura del DTM extraído del propio plot mediante filtrado de puntos clasificados como suelo. Operar en HAG produce *invarianza al relieve*: dos árboles vecinos sobre una ladera mantienen la misma altura efectiva en HAG aunque sus picos en Z absoluto difieran varios metros, evitando que la voxelización los separe artificialmente. Es la práctica estándar en segmentación volumétrica forestal en terreno con relieve no trivial.

Procedimiento. El segmentador se aplica sobre el subconjunto de puntos clasificados como árbol en la Etapa 1 (o sobre la nube completa de puntos válidos en el Flujo A) y sigue cuatro pasos:

Paso 1 — Voxelización en (X, Y, HAG) . Los puntos seleccionados se voxelizan en una grilla 3D regular en la que el eje vertical es HAG en lugar de Z absoluto. El tamaño de vóxel es parámetro del grid search.

Paso 2 — Suavizado gaussiano 3D. Sobre la grilla de densidad por vóxel se aplica un filtro gaussiano 3D para eliminar fluctuaciones de alta frecuencia debidas a ramas individuales y ruido de medición. El parámetro σ es parte del grid search.

Paso 3 — Semillas sobre la envolvente superior del volumen. Para cada columna (x, y) del grid 3D se identifica el vóxel ocupado más alto, construyendo un mapa de alturas local derivado del propio grid (equivalente a un CHM computado internamente sobre la nube suavizada). Sobre este mapa se aplica **peak_local_max** con separación mínima entre picos dada por $min_crown_radius_m / voxel_size$ y umbral absoluto igual a min_tree_height , obteniendo una semilla (i_x, i_y, top_z) por árbol candidato, donde top_z es el índice del vóxel más alto de la columna. Las semillas con altura efectiva menor a min_tree_height se descartan. El parámetro $min_crown_radius_m$ es parte del grid search; min_tree_height y $min_points_per_tree$ se mantienen fijos.

Paso 4 — Watershed volumétrico 3D. A partir de las semillas se ejecuta **skimage.segmentation.watershed** sobre el grid 3D de densidad suavizada invertida ($-density_smooth$), con máscara **density_grid** > 0 . El operador produce etiquetas de instancia por *vóxel tridimensional*; cada punto del plot recibe la etiqueta del vóxel 3D que lo contiene. A diferencia de un watershed 2D sobre CHM, la asignación de puntos sigue crestas de densidad volumétricas y no la proyección horizontal, lo que separa correctamente troncos adyacentes y puntos de sotobosque que en un watershed 2D caerían bajo la proyección de una copa vecina. Los segmentos con menos de $min_points_per_tree$ puntos se descartan y sus puntos reciben la etiqueta 0 (fondo), que no contamina la evaluación de instancia.

Calibración por grid search independiente por flujo. Los parámetros del segmentador se calibran de forma independiente para cada flujo mediante grid search exhaustivo sobre el split val, maximizando el F1 de instancia (umbral IoU 3D ≥ 0.5 , ver Sección 5). La calibración independiente es necesaria porque los puntos de entrada al watershed difieren entre flujos: el Flujo A recibe todos los puntos válidos incluyendo suelo y vegetación baja, mientras que los Flujos B y C reciben únicamente puntos clasificados como árbol y filtrados por $HAG \geq 0.5$ m. Esta diferencia en la distribución de puntos de entrada justifica un óptimo distinto para cada flujo.

El espacio de búsqueda cubre las tres dimensiones que más afectan la formación de semillas y la fragmentación de copas. Los parámetros restantes (min_tree_height y $min_points_per_tree$) se mantienen fijos en sus valores por defecto para mantener la dimensionalidad del grid manejable.

El total de combinaciones por flujo es $3 \times 3 \times 4 = 36$. Cada combinación se evalúa sobre todos los plots val, y el mejor parámetro por flujo es el que

Table 5. Espacio de búsqueda del grid search del Watershed 3D y parámetros óptimos por flujo (calibrados sobre el split val maximizando F1 de instancia con IoU 3D ≥ 0.5).

Parámetro	Espacio	Flujo A	Flujo B	Flujo C
voxel_size (m)	{0.1, 0.2, 0.3}	[X.X]	[X.X]	[X.X]
gaussian_sigma	{0.3, 0.5, 1.0}	[X.X]	[X.X]	[X.X]
min_crown_radius_m	{0.5, 1.0, 1.5, 2.0}	[X.X]	[X.X]	[X.X]
min_tree_height (m)	Fijo: 2.0	2.0	2.0	2.0
min_points_per_tree	Fijo: 50	50	50	50

maximiza el F1 de instancia medio. Los resultados del grid search se guardan en `output/results/grid_search_best_params.csv` y son consultados por los pipelines en su etapa de instancia para garantizar reproducibilidad y trazabilidad de la calibración.

5 Protocolo de Evaluación

5.1 Métricas de segmentación semántica

La calidad del preprocesado semántico se evalúa a nivel de punto individual comparando la etiqueta predicha (árbol / no-árbol) con la etiqueta ground truth derivada del treeID de FOR-instance (treeID $> 0 \rightarrow$ árbol, excluyendo out-points).

tree IoU — Intersection over Union de la clase árbol:

$$\text{tree IoU} = \frac{TP}{TP + FP + FN} \quad (2)$$

seg mIoU — mean IoU promediado sobre ambas clases:

$$\text{seg mIoU} = \frac{1}{2} \left(\frac{TP}{TP + FP + FN} + \frac{TN}{TN + FP + FN} \right) \quad (3)$$

seg F1 — F1 *binario* de la clase positiva (árbol):

$$\text{seg F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

donde $\text{Precision} = TP/(TP + FP)$ y $\text{Recall} = TP/(TP + FN)$, computados exclusivamente sobre la clase árbol. En la implementación esto corresponde a `sklearn.metrics.f1_score` con `average='binary'` y `pos_label=1`. No se reporta macro-F1 porque el interés experimental es explícitamente la capacidad de cada método para aislar la clase árbol, no su balance global entre clases.

5.2 Métricas de segmentación de instancias

La calidad de la segmentación de instancias se evalúa a nivel de árbol individual comparando, para cada plot, el conjunto de instancias predichas $\{P_i\}_{i=1}^M$ contra el conjunto de instancias ground truth $\{G_j\}_{j=1}^N$ derivadas del treeID de FOR-instance. Cada instancia es el subconjunto de puntos asignado a un mismo identificador. El emparejamiento sigue el protocolo establecido por ForAINet [15] y SegmentAnyTree [16]: a cada instancia GT se le empareja como máximo una instancia predicha, eligiendo de forma *greedy* la que maximiza el IoU 3D punto a punto.

IoU 3D punto a punto. Para cada par (P_i, G_j) se define

$$\text{IoU}_{3D}(P_i, G_j) = \frac{|P_i \cap G_j|}{|P_i \cup G_j|}, \quad (5)$$

donde $|\cdot|$ denota número de puntos.

Matching greedy por IoU. Dadas las predicciones $\{P_i\}$ y los GT $\{G_j\}$ de un plot, el procedimiento de emparejamiento es el siguiente:

1. Se construye la matriz completa de IoU 3D $\text{IoU}(P_i, G_j) = |P_i \cap G_j| / |P_i \cup G_j|$.
2. Se descartan todos los pares con $\text{IoU} < \tau$ con $\tau = 0,5$.
3. Los pares restantes se ordenan por IoU *descendente*. En caso de empate exacto se desempata por el orden natural de los identificadores (primero por menor *pred_id*, después por menor *gt_id*) para garantizar determinismo.
4. Se recorre la lista ordenada: un par (P_i, G_j) se acepta como verdadero positivo (TP) si y sólo si ni P_i ni G_j han sido asignados previamente. Tras aceptar el par, ambos se marcan como usados y no vuelven a participar en emparejamientos posteriores.
5. Al terminar, las predicciones sin pareja cuentan como falsos positivos (FP) y los GT sin pareja como falsos negativos (FN).

Este criterio garantiza que cada instancia predicha se asocia a lo sumo con un GT y viceversa, es determinístico entre corridas, y sigue el protocolo reportado en ForAINet [15] y SegmentAnyTree [16]. El umbral de $\tau = 0,5$ es el estándar adoptado en la literatura de segmentación de instancias forestales.

A partir de TP, FP y FN se computan precisión, recall y F1 basados en el greedy matching 1-a-1 descrito arriba. Además reportamos *coverage* (cobertura), definido como la fracción de árboles GT que reciben *al menos una predicción* con $\text{IoU}_{3D} \geq \tau$, **sin imponer matching único**. A diferencia del recall, coverage no penaliza la sobre-segmentación: si dos predicciones disjuntas solapan el mismo GT por encima del umbral, ambas contribuyen a cubrir ese GT aunque solo una pueda quedar emparejada en el greedy 1-a-1. Por construcción $\text{Coverage} \geq \text{Recall}$, y la brecha $\text{Coverage} - \text{Recall}$ es un indicador directo de fragmentación de copas por el segmentador:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN} \quad (6)$$

Table 6. Resultados agregados sobre el conjunto de test de FOR-instance V1. Las métricas de instancia usan $\text{IoU}_{3D} \geq 0.5$ como umbral de TP.

Flujo	sem mIoU	tree IoU	sem F1	inst Prec	inst Recall	inst F1	Coverage
A — Sin preproc.	—	—	—	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]
B — Random Forest	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]
C — PointNet++ MSG	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]	[X.XXX]

Table 7. tree IoU semántico por sitio y flujo sobre el conjunto de test (no aplica al Flujo A, que no clasifica semánticamente).

Sitio	GT Trees	tree IoU (B — RF)	tree IoU (C — PN++)
CULS	20	[X.XXX]	[X.XXX]
NIBIO	161	[X.XXX]	[X.XXX]
RMIT	64	[X.XXX]	[X.XXX]
SCION	43	[X.XXX]	[X.XXX]
TUWIEN	35	[X.XXX]	[X.XXX]

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{Coverage} = \frac{|\{G_j : \exists P_i \text{ con } \text{IoU}_{3D} \geq 0.5\}|}{|\{G_j\}|} \quad (7)$$

Las métricas se computan por plot y luego se promedian sobre los plots del split, reportando también el conteo agregado de árboles GT y predichos por flujo.

5.3 Justificación del uso de ambas tareas en cascada

Evaluar únicamente la segmentación semántica no permite concluir sobre la utilidad del preprocesado para la segmentación de instancias: un pipeline puede tener tree IoU alto pero F1 de instancia bajo si el segmentador no separa copas adyacentes, independientemente de la calidad de la clasificación punto a punto. Evaluar únicamente el F1 de instancia no permite identificar en cuál de las dos etapas se concentra el error. Reportar ambas tareas en cascada permite responder la pregunta central del trabajo: *és el preprocesado semántico o el segmentador el cuello de botella del pipeline?*

Table 8. F1 de instancia ($\text{IoU } 3\text{D} \geq 0.5$) por sitio y flujo sobre el conjunto de test.

Sitio	GT Trees	inst F1 (A)	inst F1 (B — RF)	inst F1 (C — PN++)
CULS	20	[X.XXX]	[X.XXX]	[X.XXX]
NIBIO	161	[X.XXX]	[X.XXX]	[X.XXX]
RMIT	64	[X.XXX]	[X.XXX]	[X.XXX]
SCION	43	[X.XXX]	[X.XXX]	[X.XXX]
TUWIEN	35	[X.XXX]	[X.XXX]	[X.XXX]

Table 9. Parámetros óptimos del Watershed 3D calibrados por grid search sobre el split val maximizando F1 de instancia.

Parámetro	Flujo A	Flujo B (RF)	Flujo C (PN++)
voxel_size (m)	[X.X]	[X.X]	[X.X]
gaussian_sigma	[X.X]	[X.X]	[X.X]
min_crown_radius_m	[X.X]	[X.X]	[X.X]

6 Resultados

6.1 Resultados agregados

6.2 Resultados por sitio — Segmentación semántica

6.3 Resultados por sitio — Segmentación de instancias

6.4 Parámetros óptimos del Watershed 3D por flujo

6.5 Importancia de features — Random Forest

6.6 Análisis de la relación segmentación–detección

7 Discusión

7.1 El segmentador como cuello de botella

7.2 Viabilidad de Random Forest sin GPU

7.3 Generalización de PointNet++ entre tipos de bosque

7.4 Limitaciones

1. **Tamaño del conjunto de test.** El conjunto de test oficial de FOR-instance comprende 11 plots y 323 árboles GT. El número de árboles por sitio es limitado (20–161), lo que introduce varianza en las estimaciones por sitio, especialmente en CULS (20 árboles) y TUWIEN (35 árboles).
2. **Entrenamiento único de PointNet++.** Por restricciones computacionales, PointNet++ se evalúa con una única configuración de hiperparámetros. No se puede descartar que una búsqueda exhaustiva mejore su rendimiento.

Table 10. Top-5 features por importancia en el clasificador Random Forest.

Rank	Feature	Importancia
1	[PENDIENTE]	[X.XXXX]
2	[PENDIENTE]	[X.XXXX]
3	[PENDIENTE]	[X.XXXX]
4	[PENDIENTE]	[X.XXXX]
5	[PENDIENTE]	[X.XXXX]

3. **Especificidad del segmentador.** Las conclusiones sobre el cuello de botella son específicas al Watershed 3D volumétrico empleado. Otros segmentadores de instancias forestales — por ejemplo clustering jerárquico 3D, métodos basados en grafos o redes neuronales end-to-end como ForAINet [15] y SegmentAnyTree [16] — podrían mostrar una relación diferente entre preprocesado semántico y calidad de instancia.
4. **Datos ULS de alta densidad.** Los resultados son específicos a nubes de puntos ULS con densidades en el rango 454–10 156 pts/m² (ver §3). La transferencia a datos ALS de baja densidad (~ 5 pts/m²) no está garantizada.
5. **Dataset de un solo benchmark.** La evaluación se realiza exclusivamente sobre FOR-instance V1. La generalización a otros datasets ULS o de LiDAR denso requiere validación adicional.

7.5 Trabajo futuro

- Comparación directa con segmentadores end-to-end (ForAINet [15], SegmentAnyTree [16]) para situar el pipeline de dos etapas frente a alternativas que aprenden conjuntamente semántica e instancia.
- Extensión a FOR-instanceV2 [10] para evaluar generalización a un dataset más amplio.
- Evaluación sobre datos ALS de baja densidad (ej. NeonTreeEvaluation [6]) para determinar si el rol relativo del preprocesado semántico vs. el segmentador cambia en regímenes de baja densidad.
- Exploración de features radiométricos adicionales (número de retorno, RGB cuando esté disponible) en el clasificador RF, y evaluación de su impacto sobre la segmentación semántica y la calidad de instancia.

8 Conclusiones

References

1. Puliti, S., et al.: FOR-instance: a UAV laser scanning benchmark dataset for semantic and instance segmentation of individual trees. arXiv preprint arXiv:2309.01279 (2023). DOI: 10.5281/zenodo.8287792

2. Qi, C.R., Yi, L., Su, H., Guibas, L.J.: PointNet++: Deep hierarchical feature learning on point sets in a metric space. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 5099–5108 (2017)
3. Qi, C.R., Su, H., Mo, K., Guibas, L.J.: PointNet: Deep learning on point sets for 3D classification and segmentation. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 652–660 (2017)
4. Yan, X.: `Pointnet_Pointnet2_pytorch`: PyTorch implementation of PointNet and PointNet++. GitHub repository, https://github.com/yanx27/Pointnet_Pointnet2_pytorch (2019). Accedido 2026-04
5. Breiman, L.: Random Forests. Machine Learning **45**(1), 5–32 (2001)
6. Weinstein, B.G., Marconi, S., Bohlman, S., Zare, A., White, E.: A benchmark dataset for individual tree crown delineation in co-registered airborne RGB, LiDAR and hyperspectral imagery. Methods in Ecology and Evolution **12**(10), 1847–1856 (2021)
7. Silva, C.A., Hudak, A.T., Vierling, L.A., et al.: Imputation of individual longleaf pine (*Pinus palustris* Mill.) tree attributes from field and LiDAR data. Canadian Journal of Remote Sensing **42**(5), 554–573 (2016)
8. Popescu, S.C., Wynne, R.H., Nelson, R.F.: Estimating plot-level tree heights with lidar: local filtering with a canopy-height based variable window size. Computers and Electronics in Agriculture **37**(1–3), 71–95 (2002)
9. Dalponte, M., Coomes, D.A.: Tree-centric mapping of forest carbon density from airborne laser scanning and hyperspectral data. Methods in Ecology and Evolution **7**(10), 1236–1245 (2016)
10. Xiang, B., et al.: FOR-instanceV2: A large-scale benchmark for 3D forest point cloud instance segmentation. (2025)
11. Weinmann, M., Jutzi, B., Hinz, S., Mallet, C.: Semantic point cloud interpretation based on optimal neighborhoods, relevant features and efficient classifiers. ISPRS Journal of Photogrammetry and Remote Sensing **105**, 286–304 (2015)
12. Weinmann, M., Jutzi, B., Mallet, C., Weinmann, M.: Geometric features and their relevance for 3D point cloud classification. ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences **IV-1/W1**, 157–164 (2017)
13. Li, X., et al.: Point cloud classification in forestry using geometric features and Random Forest. Remote Sensing **15**(8), 2103 (2023)
14. Bremer, M., et al.: Feature importance analysis for LiDAR forest classification. ISPRS Journal of Photogrammetry and Remote Sensing (2023)
15. Henrich, J., et al.: ForAINet: Deep learning for forest point cloud semantic and instance segmentation. (2024)
16. Wielgosz, M., et al.: SegmentAnyTree: A sensor and platform agnostic deep learning model for tree segmentation using laser scanning data. (2024)
17. Wang, D., et al.: Semantic segmentation of forest point clouds: Combining deep learning with height-normalized intensity features. Remote Sensing **15**(8) (2023)

Appendix A — Configuración de experimentos y reproducibilidad

Table 11. Arquitectura detallada de PointNet++ MSG (Flujo C). Los radios operan en el espacio normalizado del sample (tras dividir por el radio máximo del bloque de 8,192 puntos). En cada capa SA, los canales de salida de cada grupo MSG se concatenan para formar el canal de entrada de la capa siguiente. Las capas FP usan interpolación 3-NN ponderada por $1/d$.

Capa	npoint	radios	nsamples	MLPs por grupo	Salida
<i>Encoder (Set Abstraction MSG)</i>					
SA1	1,024	[0.1, 0.2]	[16, 32]	[16,16,32], [32,32,64]	96 ch
SA2	256	[0.2, 0.4]	[16, 32]	[64,64,128], [64,96,128]	256 ch
SA3	64	[0.4, 0.8]	[16, 32]	[128,128,256], [128,128,256]	512 ch
SA4	16	[0.8, 1.6]	[16, 32]	[256,256,512], [256,384,512]	1,024 ch
<i>Decoder (Feature Propagation)</i>					
FP4	—	—	—	[512, 512]	512 ch
FP3	—	—	—	[512, 256]	256 ch
FP2	—	—	—	[256, 128]	128 ch
FP1	—	—	—	[128, 128]	128 ch
<i>Cabeza de clasificación</i>					
Conv1d(128 $\rightarrow$ 64) + BN + ReLU + Dropout($p=0,5$) $\rightarrow$ Conv1d(64 $\rightarrow$ 2) + log_softmax					

Table 12. Detalles de reproducción.

Componente	Detalle
Dataset	FOR-instance V1 (DOI: 10.5281/zenodo.8287792)
Split	Oficial dev/test sin modificaciones
Hardware PointNet++	Apple M4 Pro (memoria unificada), batch_size = 16, bf16 , 100 epochs; tiempo total: [pendiente]
Hardware RF	CPU, ~16 min entrenamiento
Framework PointNet++	PyTorch (sin extensiones CUDA)
Framework RF	scikit-learn
Código	[PENDIENTE: URL repositorio]
Semilla aleatoria	42 (global: random , numpy , torch ; ver §3.3). La reproducibilidad bit-exact del entrenamiento